

SE3110 Quality Management in Software Engineering

Year 3 Semester 1 2026

Software Testing and Quality Evaluation

TalentFlow AI

Intelligent Recruitment and Employee Management Platform

SE3090 integrated system | Group SE012

Student	Registration number	Primary component
S.H.V.A Rosa	IT24100197	Company and Job Management
Hewapathirana S.A	IT24100097	Candidate and Application Management
Madhushan W.M.G	IT24100715	Interview and Hiring Management
Dewmini G.D.C.P	IT24102755	Employee and Onboarding Management

Assignment deadline 8 October 2026

Verification and report revision 9 October 2026

Repository https://github.com/sishanhewa/SE3090_SE012

This report records executable tests, real application screenshots, tool-generated results, defects and retesting.
The test case register and evidence bundle accompany the report.

Contents

1 Executive Summary	3
2 System Scope and Quality Risks	4
3 Test Plan	5
4 Test Environment and Reproduction	7
5 Backend and Database Testing	9
6 React Web Application Testing	12
7 Flutter Mobile Application Testing	15
8 Integrated Workflow and API Security	16
9 Agentic AI Evaluation	18
10 Performance and Stress Testing	20
11 Defect Report and Retesting	23
12 Test Execution Summary and Coverage	25
13 Individual Responsibilities and Viva Demonstrations	26
14 GitHub Actions and Contribution Evidence	28
15 AI Assistance Declaration	30
16 Conclusion	31
Appendix A Complete Test Case Register	32
Appendix B Evidence Inventory and References	47

The complete test case register contains inputs or execution steps, expected outcomes, actual outcomes and status. Raw logs and rerun scripts are provided in the supporting bundle.

1 Executive Summary

TalentFlow AI was evaluated across its ASP.NET Core API, PostgreSQL database, React web application, Flutter mobile application and Python agentic AI subsystem. The final local verification executed 176 automated cases and checks: 56 backend, 25 React, 16 Flutter, 52 AI, 22 live API workflow and security checks, and five PostgreSQL checks. All 176 passed after the verified fixes. The load and stress scenarios are reported separately because their HTTP checks are repeated measurements, not additional unique functional test cases.

Live integration testing exposed a reproducible application withdrawal failure that service mocks did not reveal. Two prompt-injection detection bypasses were reproduced and repaired; Unicode obfuscation now has an explicit blocking assertion. The report includes baseline failures, implementation changes, regression results and tool outputs.

The 50-user load scenario produced 4,795 requests with no failed requests and a p95 duration of 59.02 ms. The 200-user authentication stress scenario produced 28,328 requests with no failed requests and a p95 duration of 892.00 ms. These are local measurements on a seeded dataset, not a production capacity guarantee.

The passing result is bounded by test scope. Instrumented backend line coverage is 13.70%; external Gemini answer quality, Google Calendar behavior, production hosting, a complete hire-to-onboarding UI journey, mobile device hardware and broad penetration testing were not established by this run.

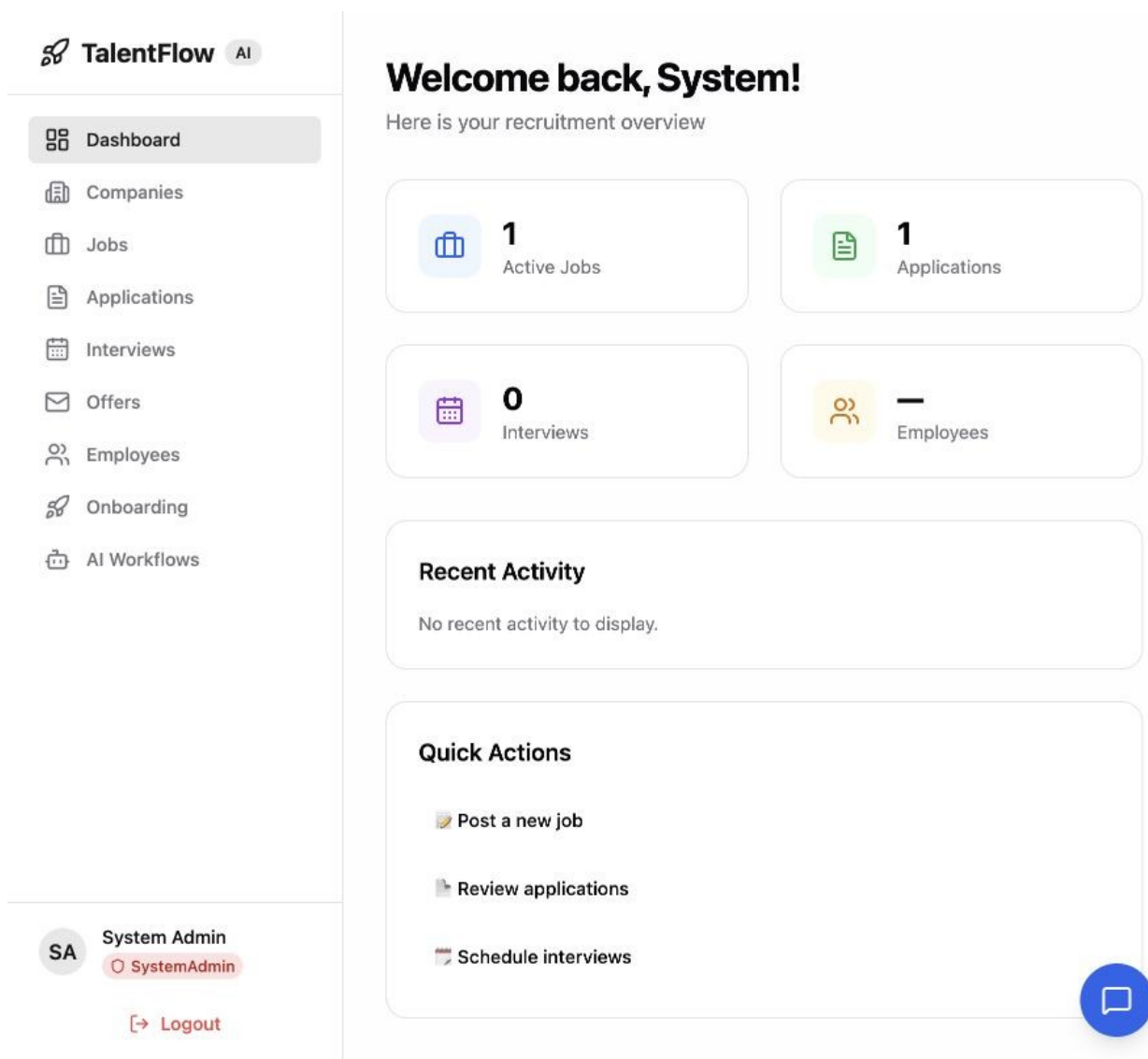


Figure 1 TalentFlow admin dashboard after signing in to the isolated verification system

2 System Scope and Quality Risks

2 1 Architecture and implemented technology

The React administration interface and Flutter candidate interface use a shared ASP.NET Core REST API. Entity Framework Core with Npgsql persists system data in PostgreSQL. The Python FastAPI subsystem coordinates candidate analysis, validation and interview planning. AI recommendations remain subject to workflow review; tests use deterministic data and controlled substitutes for external dependencies.

Layer	Implemented technology and role
Backend	ASP.NET Core .NET 8; application, domain and infrastructure layers; Identity and JWT authentication
Database	PostgreSQL 16 with EF Core migrations and relational constraints
Web	React and TypeScript; Vite; Zustand; React Query; React Hook Form and Zod
Mobile	Flutter and Dart; production screens use Provider, package:http and GoRouter. Riverpod and Dio are declared dependencies but are not the exercised login implementation.
AI	FastAPI, LangGraph, Gemini client, Pydantic schemas and an agent tool allowlist

2 2 Scope and exclusions

In scope are service validation and state changes, authentication UI, mobile login and job list behavior, routing guards, candidate application submission and withdrawal through the real API, database constraints, AI tool permissions and prompt safety, and local performance. The integrated acceptance scenario completes the application submission, staff review and candidate withdrawal lifecycle using the same persisted application ID.

Service-to-EF InMemory tests are described as persistence or business-logic tests, not proof of PostgreSQL constraint behavior. PostgreSQL constraints are separately exercised through psql. The existing .NET integration test project contains no discoverable tests; the live API script provides the executed API integration evidence.

Risk	Impact	Verification
Unauthorized access or role escalation	Candidate or company data exposed	TC-SEC-001 to TC-SEC-009; malformed token, role and owner checks
Invalid or duplicate applications	Incorrect recruitment state	ApplicationService tests; live duplicate check; PostgreSQL unique constraint
History persistence failure	Action fails or audit trail is missing	New repository regression and live withdrawal retest
Unsafe AI recommendation or tool use	Unreviewed changes or data access	Tool allowlist tests, scoring validator tests, coordinator approval tests
Slow API under concurrency	Poor usability and failed requests	k6 50-user load and 200-user authentication stress scenarios
Misleading test evidence	Results cannot be explained or reproduced	Raw results, per-case register, explicit test boundaries and rerun commands

3 Test Plan

3 1 Objectives and strategy

The strategy starts with isolated service and component tests, then verifies selected behavior against the actual API and PostgreSQL database. Normal, invalid, boundary and failure inputs are selected around the system risks. Functional tests assert status codes, returned values, visible UI, persisted state or exceptions; performance tests apply configured latency and error-rate thresholds.

Area	Tool and execution	Acceptance criterion	Assigned owner
Backend	xUnit, Moq, FluentAssertions and EF InMemory	All 56 final assertions pass	Own component members
PostgreSQL	psql with exception assertions and rollback	Five checks pass with exit code 0	Lead shared verification
React	Vitest, React Testing Library and JSDOM	25 tests pass; web build succeeds	Lead
Flutter	flutter_test with fake auth and MockClient	16 production widget and router tests pass	Interview and Hiring member
API integration and security	Python standard-library HTTP test script	22 checks pass; persisted state and owner isolation hold	Lead shared verification
AI	pytest and pytest-asyncio	52 deterministic cases pass; no xfail masks remain	Lead and safety owner
Performance	k6 load and authentication stress scripts	Each configured latency and error threshold passes	Company and Job member

3 2 Entry and exit criteria

Entry requires the correct project checkout, installed dependencies, a seeded isolated PostgreSQL database, a running API, known development test accounts and a traceable source snapshot. Exit requires completion of selected executable cases, investigation of failures, retesting of fixes, readable screenshot evidence, a case register and documented residual risks. The local exit criteria were met for the selected scope. Remote component delivery depends on the account workflow recorded in Section 14.

3 3 Verification sequence and test data

Phase on 9 October 2026	Activity	Recorded outcome
Review	Compare brief, supplied draft and Group 020 sample; inspect source and CI	Correct scope, statistics and evidence requirements
Baseline	Run supplied backend, web, mobile and AI suites	55 backend; 25 web; 32 mobile; 55 AI passes and two AI xfails
Integration	Run candidate application lifecycle against real API and DB	Withdrawal HTTP 500 reproduced
Repair and retest	Repair history tracking and prompt safety; strengthen weak tests	56 backend, 25 web, 16 production mobile and 52 AI passes

Phase on 9 October 2026	Activity	Recorded outcome
Quality evaluation	Run live security, database, k6 and coverage checks	22 API and five DB checks pass; both k6 scenarios pass
Documentation	Record cases, evidence, limitations and contribution workflow	Revised report and supporting bundle

Synthetic candidate addresses use the example.test domain. Fresh identifiers prevent duplicate fixture collisions. API tests retain JWTs in memory and omit their values from evidence. Database checks roll back their mutations. The verification database is talentflow_se3110_verification on the local Docker PostgreSQL instance; the existing project database was not used for these mutations.

4 Test Environment and Reproduction

Item	Observed configuration
Operating environment	macOS Apple Silicon
.NET SDK	8.0.423; target framework net8.0
PostgreSQL	16.15 in Docker; host port 5433
Node runtime	24.19.0 for local execution; CI configured for Node 22
React runner	Vitest 3.2.7 with JSDOM; tests use @testing-library/jest-dom
Flutter and Dart	Flutter 3.38.3; Dart 3.10.1
AI Python	Python 3.12.4 in project virtual environment; CI Python 3.13
k6	Official macOS ARM64 release 2.3.0
Verification endpoints	API http://127.0.0.1:5155; web http://127.0.0.1:5178 via API proxy

4 1 Rerun commands

From the repository root, use the following commands after installing dependencies and starting the isolated API. For API execution, set `ConnectionStrings__DefaultConnection` to the verification database and `ASPNETCORE_ENVIRONMENT` to `Development`. The development seeder applies migrations and creates sample data. Use a fresh disposable database when reproducing the integration tests.

```
dotnet test backend/tests/TalentFlow.UnitTests --logger "trx;LogFileName=backend.trx" --collect:"XPlat Code Coverage"
```

```
cd frontend/react-app && npm ci && npm run test -- --run --reporter=verbose
```

```
cd frontend/flutter_app && flutter pub get && flutter test --reporter expanded
```

```
cd agentic-ai && pip install -r requirements.txt && pytest tests/ -v --junitxml=test-results.xml
```

```
python docs/testing/scripts/live_api_tests.py --base-url http://127.0.0.1:5155/api
```

```
docker exec -i talentflow-postgres psql -U talentflow_user -d talentflow_se3110_verification <  
docs/testing/scripts/database_checks.sql
```

```
k6 run -e SUMMARY_PATH=load-summary.json performance/k6/load_test_api.js
```

```
k6 run -e SUMMARY_PATH=stress-summary.json performance/k6/stress_test_auth.js
```

`TEST_ADMIN_EMAIL` and `TEST_ADMIN_PASSWORD` can override the development fixture credentials. Performance scripts use `BASE_URL` for the API base and `SUMMARY_PATH` for the generated JSON. Do not point these mutation and load scripts at a production database. The evidence inventory includes the scripts and exact result exports used for this report.

Test configuration and source inventory

TalentFlow AI | SE3110 | Verification 9 October 2026
Source: Actual repository paths and vitest.config.ts

```
Backend: backend/tests/TalentFlow.UnitTests/Services
  JobServiceTests.cs | CompanyServiceTests.cs
  ApplicationServiceTests.cs | ApplicationHistoryPersistenceTests.cs
  InterviewServiceTests.cs | OfferServiceTests.cs
  EmployeeServiceTests.cs | OnboardingServiceTests.cs
  WorkflowServiceTests.cs | CandidateScoringServiceTests.cs
```

React configuration

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import path from 'path';
```

```
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(import.meta.dirname, './src'),
    },
  },
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['./src/__tests__/setup.ts'],
    css: true,
    include: ['src/**/*.test.ts', 'src/**/*.spec.ts', 'src/**/*.tsx'],
  },
});
```

```
AI: agentic-ai/tests/test_agents.py, test_cv_screening.py,
  test_prompt_injection.py, test_business_rules.py
Flutter: frontend/flutter_app/test/login_screen_test.dart,
  widget_test.dart, jobs_list_test.dart, navigation_test.dart
```

Actual exported tool output. Full results accompany the report.

Figure 2 Actual test source locations and React runner configuration

5 Backend and Database Testing

5 1 Backend service coverage

The backend tests use the Arrange Act Assert pattern. Moq isolates repositories where appropriate; EF InMemory supports selected service and persistence scenarios. Assertions check error codes and responses, validation guards, workflow decisions and persisted records. The final result is 56 passed, zero failed and zero skipped.

Test class	Executed	Primary behavior
ApplicationHistoryPersistenceTests	1	New history insertion and persisted withdrawal
ApplicationServiceTests	7	Service creation, retrieval, validation or deterministic business rules
CandidateScoringServiceTests	2	Service creation, retrieval, validation or deterministic business rules
CompanyServiceTests	8	Company CRUD, paging and minimal input
EmployeeServiceTests	1	Service creation, retrieval, validation or deterministic business rules
GoogleCalendarServiceTests	4	Service creation, retrieval, validation or deterministic business rules
InterviewServiceTests	7	Scheduling guards, status, duration and conflicts
JobServiceTests	2	Service creation, retrieval, validation or deterministic business rules
OfferServiceTests	6	Missing records and prerequisite guards
OnboardingServiceTests	9	Template and task operations; assignment ownership guards
SchedulingServiceTests	4	Service creation, retrieval, validation or deterministic business rules
WorkflowServiceTests	5	Workflow state and approval rejection persistence

Some supplied offer tests provide null dependencies and exercise an earlier prerequisite guard, rather than proving salary validation. This report does not treat those cases as a verified successful offer lifecycle. The validation ordering observation is recorded separately in the defect register.

Backend test execution and coverage

TalentFlow AI | SE3110 | Verification 9 October 2026
Source: dotnet test TalentFlow.UnitTests with TRX and XPlat Code Coverage

```
All projects are up-to-date for restore.
TalentFlow.Domain -> /Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/src/TalentFlow.Domain/bin/Debug/net8.0/TalentFlow.Domain.dll
TalentFlow.Application -> /Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/src/TalentFlow.Application/bin/Debug/net8.0/TalentFlow.Application.dll
/Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/src/TalentFlow.Infrastructure/Repositories/JobRepository.cs(41,38): warning CS8602: Dereference of a possibly
null reference. [/Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/src/TalentFlow.Infrastructure/TalentFlow.Infrastructure.csproj]
TalentFlow.Infrastructure -> /Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/src/TalentFlow.Infrastructure/bin/Debug/net8.0/TalentFlow.Infrastructure.dll
TalentFlow.UnitTests -> /Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/tests/TalentFlow.UnitTests/bin/Debug/net8.0/TalentFlow.UnitTests.dll
Test run for /Users/sishanhewapathirana/Documents/Codex/2026-10-
09/x20/work/project/backend/tests/TalentFlow.UnitTests/bin/Debug/net8.0/TalentFlow.UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (arm64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
Results File: /Users/sishanhewapathirana/Documents/Codex/2026-10-09/x20/work/evidence/backend-final/backend.trx

Passed! - Failed:    0, Passed:   56, Skipped:    0, Total:   56, Duration: 622 ms - TalentFlow.UnitTests.dll (net8.0)

Attachments:
/Users/sishanhewapathirana/Documents/Codex/2026-10-09/x20/work/evidence/backend-final/a1e83ada-cb4f-4147-8199-
e5d8a765fb8f/coverage.cobertura.xml
```

Actual exported tool output. Full results accompany the report.

Figure 3 Actual backend test output with 56 passes and generated coverage attachment

5.2 PostgreSQL constraints and persistence

Five additional checks execute against PostgreSQL, not an in-memory substitute. SQL exception handling expects the database to reject a duplicate job and candidate pair, an orphan history row and a NULL status. A join checks that the completed live withdrawal has matching history, and a savepoint test verifies rollback restores previous values. Unexpected acceptance raises an exception, and ON_ERROR_STOP turns it into a failing command.

Case	Expected and observed result
TC-DB-001	PostgreSQL unique_violation prevents duplicate application; PASS notice emitted; psql exit code 0
TC-DB-002	PostgreSQL foreign_key_violation rejects unknown ApplicationId; PASS notice emitted; psql exit code 0
TC-DB-003	PostgreSQL not_null_violation rejects NULL status; PASS notice emitted; psql exit code 0
TC-DB-004	Join finds Withdrawn application with Withdrawn history; PASS notice emitted; psql exit code 0
TC-DB-005	CoverLetter sentinel does not survive rollback to savepoint; PASS notice emitted; psql exit code 0

The isolated API applied InitialCreate and Sprint3_WorkflowAndOnboarding migrations successfully. These checks establish selected relational guarantees and rollback behavior; they do not measure migration upgrades from every historical schema or concurrent duplicate-request races.

PostgreSQL constraints and transaction results

TalentFlow AI | SE3110 | Verification 9 October 2026
Source: psql database_checks.sql

```
BEGIN
NOTICE: TC-DB-001 PASS duplicate job candidate pair rejected
NOTICE: TC-DB-002 PASS orphan history rejected
NOTICE: TC-DB-003 PASS null status rejected
NOTICE: TC-DB-004 PASS live withdrawal and history persisted together
DO
SAVEPOINT
UPDATE 3
ROLLBACK
NOTICE: TC-DB-005 PASS rollback restores prior application data
DO
ROLLBACK

                                version
-----
PostgreSQL 16.15 on aarch64-unknown-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
(1 row)

                                MigrationId
-----
20260914061956_InitialCreate
20260927060312_Sprint3_WorkflowAndOnboarding
(2 rows)
```

Actual exported tool output. Full results accompany the report.

Figure 4 Actual PostgreSQL assertion notices, rollback output, version and applied migrations

6 React Web Application Testing

Vitest executed 17 LoginPage component cases and eight authStore cases. The login component cases exercise rendering, email and password controls, validation, submission, error clearing, visible failure, pending state and navigation. AuthStore cases exercise login, logout, loading and failure state transitions with controlled API substitutes. All 25 passed, and TypeScript checking and the Vite production build completed successfully.

Component rendering uses MemoryRouter and mocked authentication behavior. These tests do not establish every protected page, every recruitment form or browser compatibility. The real browser screenshots separately verify that the administration app signs in and retrieves job and application data through the API proxy.

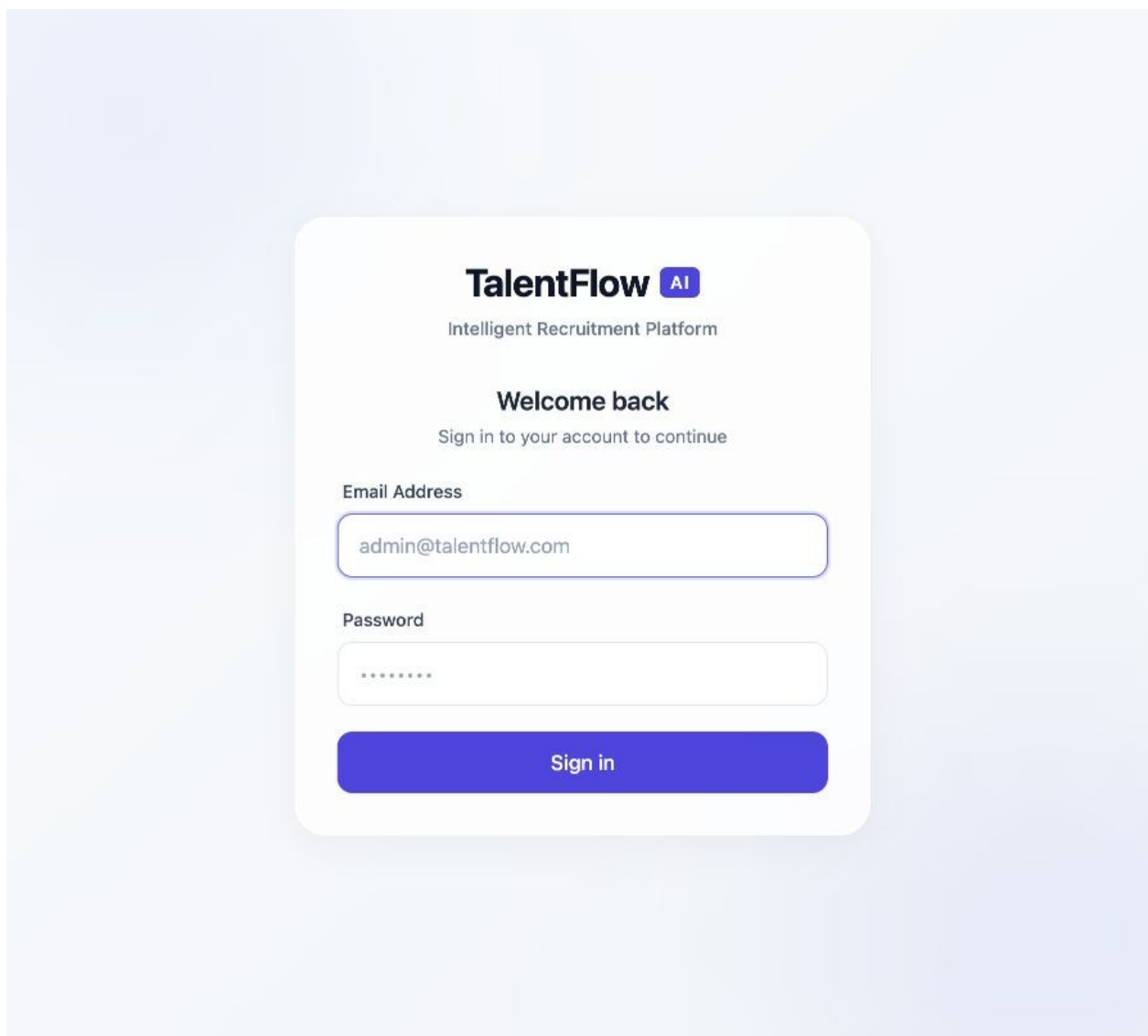


Figure 5 Real TalentFlow login page before entering credentials

React component and state tests

TalentFlow AI | SE3110 | Verification 9 October 2026
Source: Vitest verbose output

```
✓ src/__tests__/store/authStore.test.ts > authStore > should set loading state during login 0ms
✓ src/__tests__/store/authStore.test.ts > authStore > should set auth data on successful login 0ms
✓ src/__tests__/store/authStore.test.ts > authStore > should set error on failed login 0ms
✓ src/__tests__/store/authStore.test.ts > authStore > should set default error message when no server message 0ms
✓ src/__tests__/store/authStore.test.ts > authStore > should clear all auth data on logout 0ms
✓ src/__tests__/store/authStore.test.ts > authStore > should clear error when clearError is called 0ms
✓ src/__tests__/store/authStore.test.ts > authStore > should call the correct API endpoint on login 1ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should render the login form with email and password fields 74ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should render the TalentFlow AI branding 4ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should display "Welcome back" heading 3ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should have required attribute on email input 3ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should have required attribute on password input 3ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should have email type on email input 3ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should have password type on password input 2ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should update email field when user types 64ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should update password field when user types 42ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should call login function on form submit 104ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should clear error before login attempt 69ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should display error message when error exists 2ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should not display error message when no error 2ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should show loading indicator when isLoading is true 4ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should disable submit button when loading 14ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should navigate to home on successful login 115ms
✓ src/__tests__/pages/LoginPage.test.tsx > LoginPage > should not navigate when login fails 89ms
```

```
Test Files 2 passed (2)
Tests 25 passed (25)
Start at 12:12:32
Duration 1.81s (transform 104ms, setup 148ms, collect 284ms, tests 602ms, environment 850ms, prepare 123ms)
```

JUNIT report written to /Users/sishanhewapathirana/Documents/Codex/2026-10-09/x20/evidence/react.xml

Actual exported tool output. Full results accompany the report.

Figure 6 Actual Vitest output for LoginPage and authStore with 25 passes

Dashboard

Companies

Jobs

Applications

Interviews

Offers

Employees

Onboarding

AI Workflows

SA

System Admin

SystemAdmin

Logout

Jobs

Manage job postings across all companies.

+ Post a Job

All Jobs

Title	Company	Department	Status	Actions
Technical Recruiter	TechNova Solutions		Draft	View Details
Senior Backend Engineer	TechNova Solutions		Published	View Details

Figure 7 Real TalentFlow job list loaded from the verification API

7 Flutter Mobile Application Testing

The final mobile suite contains 16 executable cases against the production app: eight LoginScreen tests, five JobListScreen tests, two production GoRouter tests and the existing unauthenticated-app widget test. The supplied generic widget samples and tests that asserted locally recreated filtering logic were replaced. This deliberately reduces the headline count while increasing relevance to TalentFlow.

Production target	Cases	Assertions and dependencies
LoginScreen	8	Branding and fields, masked password, submitted credentials, failed login, pending button, retry behavior and successful GoRouter navigation. AuthProvider is replaced with a deterministic fake.
JobListScreen	5	Returned title and department, empty list, safe HTTP failure, published-job query and newest-first sort. Real widget and ApiClient execute with package:http MockClient.
GoRouter and TalentFlowApp	3	Unauthenticated startup and redirect from protected home to login; public login remains accessible.

The job-list failure case confirms the current behavior of showing No jobs found after an API error; it does not claim a dedicated network-error message exists. Mobile integration with a real device, CV file picking, onboarding and interview screens remains outside this widget test run. Flutter analysis completes with informational findings; no analyzer warnings or errors remain in the final test sources.

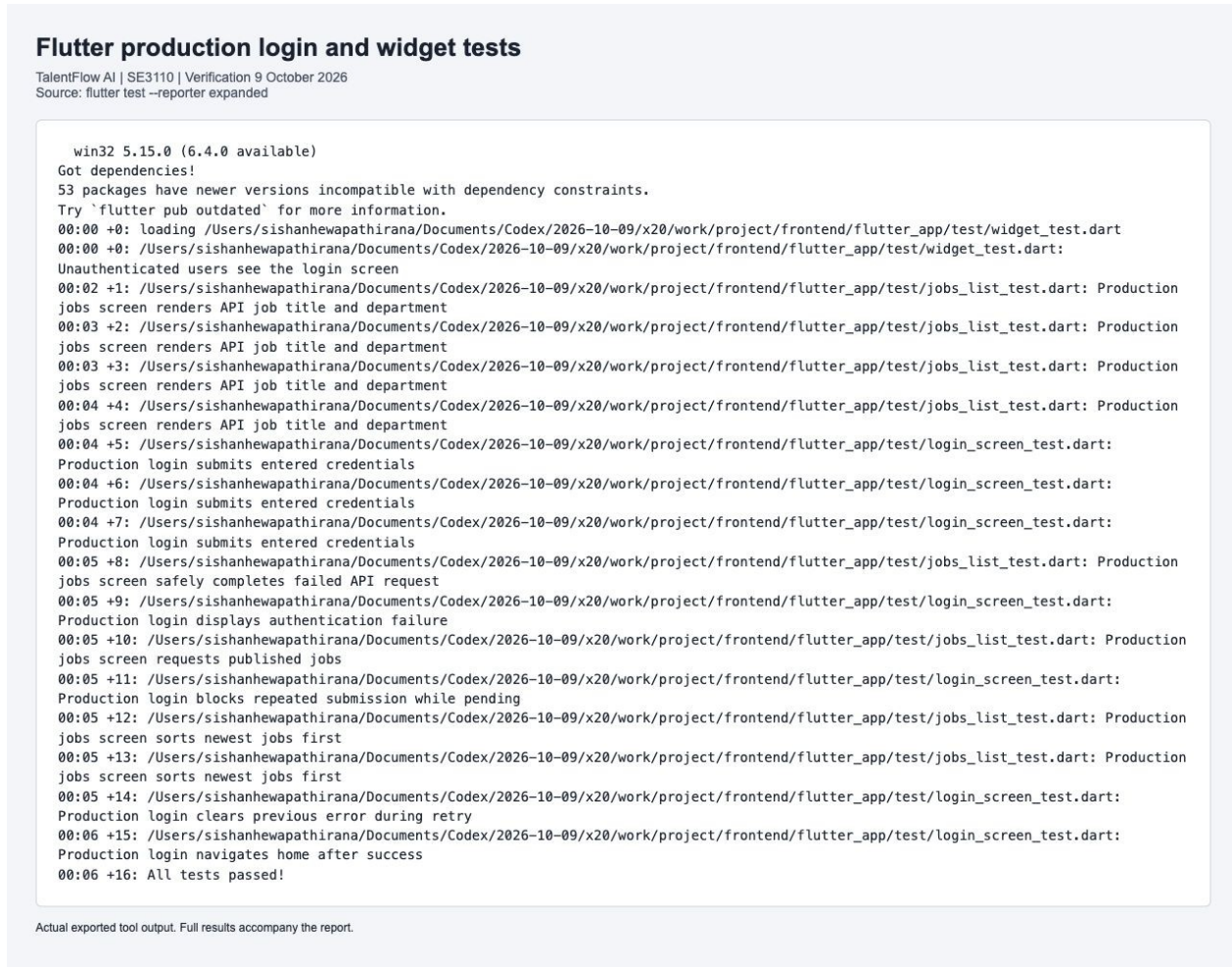


Figure 8 Actual Flutter output from the revised production widget and router suite

8 Integrated Workflow and API Security

8 1 Complete application lifecycle scenario

The live integration script executes a complete selected application lifecycle against the running API and PostgreSQL. It authenticates the seeded administrator, registers two synthetic candidates, creates their profiles and retrieves a published job. Candidate one submits an application. Candidate one and the administrator read the same persisted application ID. A duplicate submission is rejected. Candidate one withdraws the application; the administrator verifies its Withdrawn state and the audit history.

This scenario crosses authentication, candidate profiles, jobs, applications, authorization, persistence and history. The web application screenshot shows the same verification dataset with the withdrawn application visible. It is an API-level integrated workflow with a browser observation, not a claim that the entire recruitment-to-onboarding process was automated through the UI.

Stage	Expected result	Actual result
Sign in and register candidates	200 admin login; 201 candidates and profiles	Passed
Browse and submit	Published job available; 201 Submitted application	Passed
Read and prevent duplicate	200 owner and admin reads; 409 duplicate	Passed
Withdraw and verify	204 withdrawal; 200 Withdrawn read and history	Passed after DEFECT-004 repair

Live API workflow and security checks

TalentFlow AI | SE3110 | Verification 9 October 2026
Source: docs/testing/scripts/live_api_tests.py

```
TC-LIVE-001 PASS Seeded admin authenticates: HTTP 200
TC-SEC-001 PASS Protected companies reject anonymous access: HTTP 401
TC-SEC-002 PASS Malformed JWT is rejected: HTTP 401
TC-SEC-003 PASS Wrong password is rejected: HTTP 401
TC-SEC-004 PASS Public staff-role escalation is rejected: HTTP 400
TC-SEC-005 PASS Malformed login input is rejected: HTTP 400
TC-LIVE-002 PASS Register isolated candidate 1: HTTP 201
TC-LIVE-003 PASS Create candidate profile 1: HTTP 201
TC-LIVE-004 PASS Register isolated candidate 2: HTTP 201
TC-LIVE-005 PASS Create candidate profile 2: HTTP 201
TC-SEC-006 PASS Candidate cannot create company: HTTP 403
TC-LIVE-006 PASS Public jobs can be browsed: HTTP 200
TC-LIVE-007 PASS Candidate submits to published job: HTTP 201
TC-LIVE-008 PASS Candidate sees persisted application: HTTP 200
TC-SEC-007 PASS Other candidate cannot read application: HTTP 403
TC-SEC-008 PASS Candidate cannot list staff applications: HTTP 403
TC-LIVE-009 PASS Duplicate application is rejected: HTTP 409
TC-LIVE-010 PASS Staff sees submitted application: HTTP 200
TC-LIVE-011 PASS Candidate withdraws own application: HTTP 204
TC-LIVE-012 PASS Staff sees persisted withdrawal: HTTP 200
TC-LIVE-013 PASS Withdrawal is in application history: HTTP 200
TC-SEC-009 PASS Other candidate cannot withdraw application: HTTP 403
```

Actual exported tool output. Full results accompany the report.

Figure 9 Actual live API execution output with 22 passing workflow and security checks

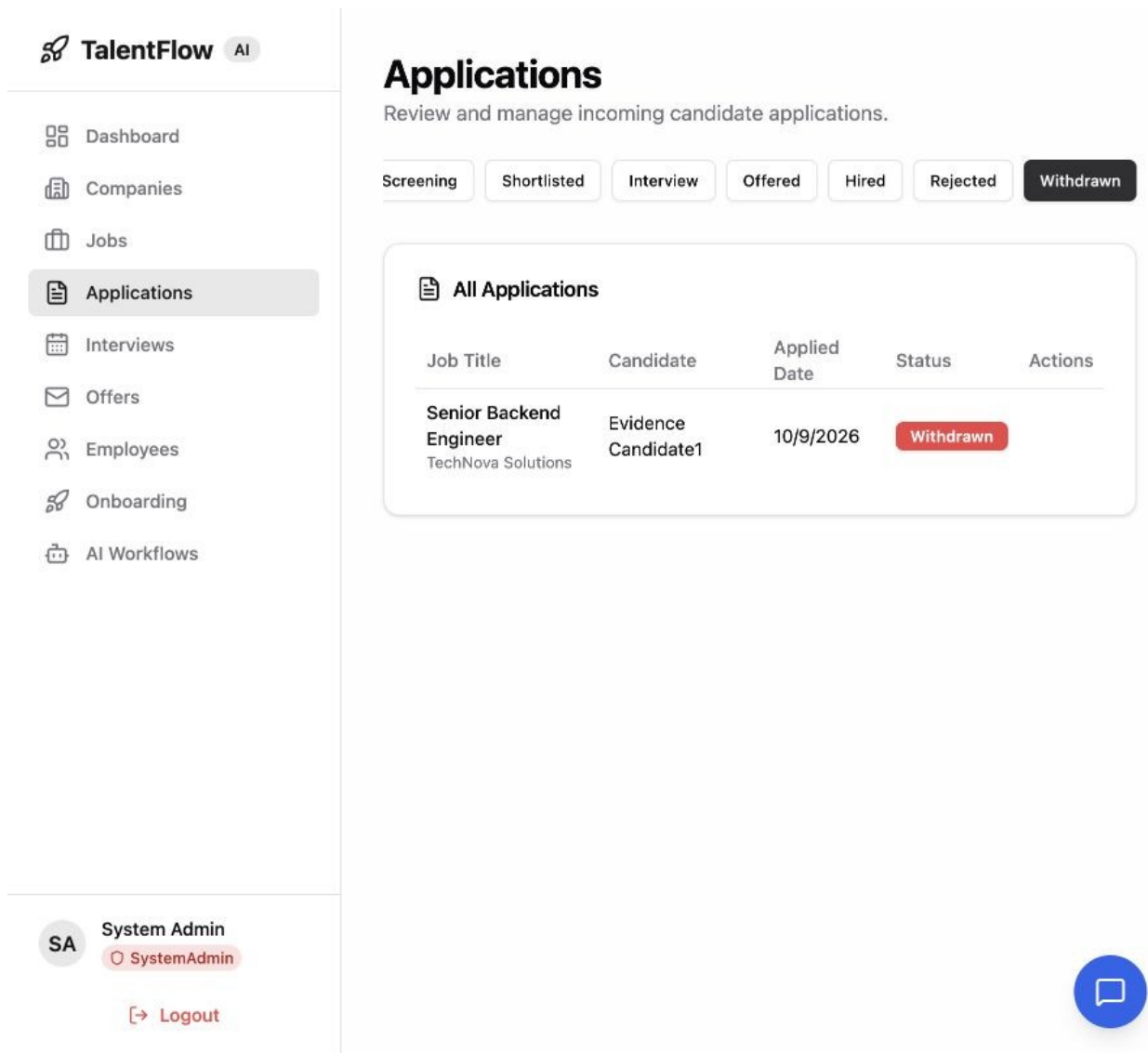


Figure 10 Withdrawn candidate application displayed in the real TalentFlow administration interface

8 2 Authentication and authorization checks

Nine checks cover anonymous access to protected companies, malformed JWT rejection, wrong-password rejection, attempted public SystemAdmin registration, malformed login input, candidate company-creation denial, another candidate reading an application, candidate access to a staff-only listing and another candidate attempting withdrawal. Expected statuses are 401, 400 or 403 depending on the control. The anonymous job listing is intentionally public and returns 200.

Security checks assert outcomes from the actual middleware and controller path. Prompt safety and AI tool restrictions are evaluated separately. This is targeted automated security verification, not an OWASP ZAP scan, penetration test, proof of SQL injection resistance or complete tenant isolation audit. No broad security scan is claimed.

9 Agentic AI Evaluation

The final AI suite has 52 passing cases. It evaluates production validation tools, tool permissions, resume parsing, coordinator recommendation and approval behavior, production scoring boundaries, Pydantic requirements and prompt detection. External HTTP and model responses are substituted in relevant unit tests, so the evaluation is deterministic and rerunnable without sending candidate data to Gemini.

9 1 Prompt safety baseline and repair

The supplied prompt tests reproduced two expected failures: new system instructions inside delimiters and a DAN jailbreak. The validator relied on substring patterns that did not include those forms. The repair adds explicit instruction and bypass patterns and normalizes compatibility characters while removing Unicode format characters before matching. The Unicode case now asserts a blocked result instead of merely checking that a result dictionary exists.



Figure 11 Supplied prompt-safety baseline showing the two documented expected failures

AI safety and scoring retest

TalentFlow AI | SE3110 | Verification 9 October 2026
Source: pytest final execution

```
work/project/agentice-ai/tests/test_business_rules.py::test_production_component_range_guards[mandatorySkillsScore--1] PASSED [ 55%]
work/project/agentice-ai/tests/test_business_rules.py::test_production_total_must_equal_component_sum PASSED [ 57%]
work/project/agentice-ai/tests/test_business_rules.py::test_workflow_result_defaults_to_human_approval PASSED [ 59%]
work/project/agentice-ai/tests/test_business_rules.py::test_candidate_analysis_requires_application_id PASSED [ 61%]
work/project/agentice-ai/tests/test_business_rules.py::test_validation_output_requires_validity_decision PASSED [ 63%]
work/project/agentice-ai/tests/test_business_rules.py::test_workflow_plan_requires_steps PASSED [ 65%]
work/project/agentice-ai/tests/test_cv_screening.py::test_attached_docx_is_read_and_used_for_skill_matching PASSED [ 67%]
work/project/agentice-ai/tests/test_cv_screening.py::test_unreadable_docx_fails_instead_of_screening_empty_evidence PASSED [ 69%]
work/project/agentice-ai/tests/test_cv_screening.py::test_screening_recommendation_comes_from_cv_checks_not_calendar_slots[True-Shortlist]
PASSED [ 71%]
work/project/agentice-ai/tests/test_cv_screening.py::test_screening_recommendation_comes_from_cv_checks_not_calendar_slots[False-
DoNotShortlist] PASSED [ 73%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_ignore_previous_instructions PASSED [ 75%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_system_prompt_override PASSED [ 76%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_role_override_attempt PASSED [ 78%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_data_exfiltration_attempt PASSED [ 80%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_instruction_injection_with_delimiters PASSED [ 82%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_jailbreak_do_anything_now PASSED [ 84%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_sql_injection_in_candidate_query PASSED [ 86%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_safe_prompt_analysis_request PASSED [ 88%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_safe_prompt_scoring_request PASSED [ 90%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_safe_prompt_interview_request PASSED [ 92%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_safe_prompt_with_technical_terms PASSED [ 94%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_empty_prompt PASSED [ 96%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_very_long_prompt PASSED [ 98%]
work/project/agentice-ai/tests/test_prompt_injection.py::TestPromptInjection::test_unicode_injection PASSED [100%]

- generated xml file: /Users/sishanhewapathirana/Documents/Codex/2026-10-09/x20/work/evidence/ai.xml -
===== 52 passed in 0.78s =====
```

Actual exported tool output. Full results accompany the report.

Figure 12 Final AI tool output after prompt repairs and meaningful scoring tests

9 2 Scoring and approval boundaries

The rewritten business-rule tests call `validate_scoring` rather than recomputing an independent scoring formula in the test. They check total scores 0, 59, 60, 74, 75 and 100, wrong recommendations, each component maximum, a negative score and an inconsistent sum. Coordinator tests independently confirm that CV checks determine recommendations and that human approval remains required. Schema cases test required fields and defaults; constructing a valid plan alone is not evidence that a generator always produces a valid plan.

Regex and substring detection are a limited defensive layer. These tests establish the observed handling of the selected strings; they do not prove resistance to every obfuscated injection, data-exfiltration request or model jailbreak. Several legacy prompt cases only assert the response shape, and are not counted as evidence of attack prevention.

10 Performance and Stress Testing

10 1 Load scenario

The load test ramps from zero to 10 users in 30 seconds, then to 50 in one minute, holds 50 for one minute and ramps down over 30 seconds. Each iteration authenticates and reads jobs, applications, companies and interviews, with pacing between requests. The fixture password was corrected to match the actual seeder, and the interviews check now requires HTTP 200 rather than accepting an unauthorized 401 response.

10 2 Authentication stress scenario

The stress test ramps through 10, 50, 100, 150 and 200 users, holds 200 for 30 seconds, then ramps down over one minute. Iterations authenticate and pause for half a second. The duration is four minutes plus any finishing iterations. This run did not reach a failure breakpoint, so a maximum sustainable capacity is not inferred.

Metric	50-user load	200-user auth stress
HTTP requests	4,795	28,328
Request rate per second	26.16	117.84
Average duration ms	16.26	358.48
p95 duration ms	59.02	892.00
Failed HTTP requests	0	0
Configured p95 limit	2000 ms	5000 ms
Configured error-rate limit	10 percent	30 percent
Threshold result	All passed	All passed

Additional load threshold	Measured p95	Result
Login below 3000 ms	64.84 ms	Passed
Job listing below 1500 ms	5.72 ms	Passed

k6 load test results

TalentFlow isolated API | Selected tool-generated values from load-summary.json

```
http_reqs: {"count": 4795, "rate": 26.16196227571621}

http_req_duration: {"p(95)": 59.0216, "avg": 16.260951407716373, "min": 1.001, "med": 5.905, "max": 113.272, "p(90)": 57.5612}

Thresholds: {"p(95)<2000": {"ok": true}}

http_req_failed: {"rate": 0, "passes": 0, "fails": 4795}

checks: {"rate": 1, "passes": 6713, "fails": 0}

errors: {"rate": 0, "passes": 0, "fails": 4795}

Thresholds: {"rate<0.1": {"ok": true}}
```

Figure 13 Actual k6 load-test metric values and threshold results

k6 stress test results

TalentFlow isolated API | Selected tool-generated values from stress-summary.json

```
http_reqs: {"count": 28328, "rate": 117.83983296261916}

http_req_duration: {"p(90)": 783.0553000000001, "p(95)": 891.99925, "avg": 358.4830235456086, "min": 51.417, "med": 297.42150000000004, "max": 1599.196}

Thresholds: {"p(95)<5000": {"ok": true}}

http_req_failed: {"rate": 0, "passes": 0, "fails": 28328}

checks: {"rate": 1, "passes": 84984, "fails": 0}

errors: {"passes": 0, "fails": 28328, "rate": 0}

Thresholds: {"rate<0.3": {"ok": true}}
```

Figure 14 Actual k6 authentication stress-test metric values and threshold results

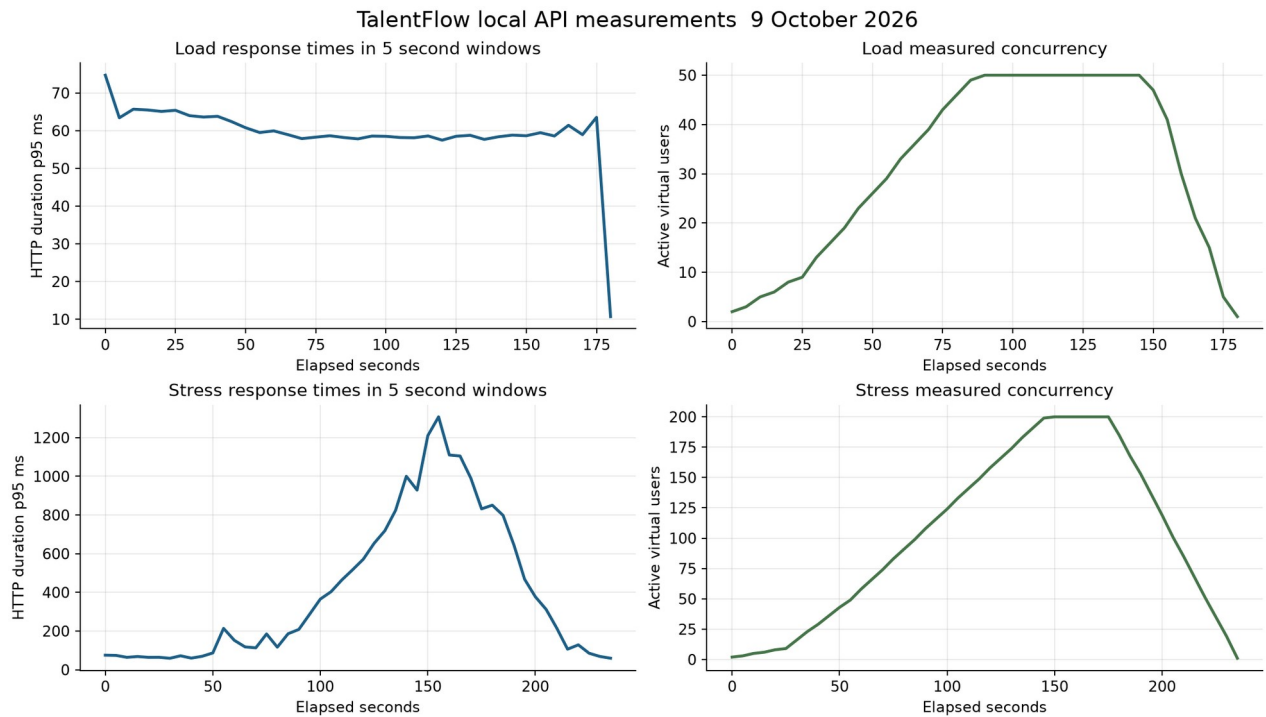


Figure 15 Measured concurrency and response-time trends from the k6 time-series exports

Measurements share one development host for the load generator and services. Database volume is small and seeded. Development logging is enabled, network travel is local and real production infrastructure is not represented. Stress thresholds allow a relatively high error rate by design, although this run observed zero. No spike, soak, recovery from process failure or external-service load result is claimed.

11 Defect Report and Retesting

ID and issue	Severity and priority	Status	Evidence and outcome
DEFECT-001 Delimiter instruction bypass	High / High	Fixed and retested	New system instructions in a code block returned safe=True; prompt retest now returns safe=False.
DEFECT-002 DAN jailbreak bypass	High / High	Fixed and retested	DAN and bypass-all-restrictions input returned safe=True; explicit patterns now block it.
DEFECT-003 Offer validation ordering	Low / Medium	Open observation	Salary/date guards follow database prerequisite checks. No production acceptance of invalid salary was demonstrated; the supplied null-context test does not prove that behavior.
DEFECT-004 Withdrawal history persistence	High / High	Fixed and retested	Live candidate withdrawal returned 500 with DbUpdateConcurrencyException; explicit Added history state now gives 204 and persists history.
DEFECT-005 React CI has no committed tests	Medium / High	Tests merged; build import fix verified	Prior workflow failed with No test files found. React tests/config are prepared on the assigned lead branch.
DEFECT-006 AI CI lint blockers	Medium / Medium	Fixed; GitHub verified	Unused imports, import placement and a dead assignment blocked pytest. Shared lint repair passed local Ruff and remote AI CI.
DEFECT-007 Flutter CI points to wrong directory	Medium / High	Fixed; GitHub verified	Workflow referenced flutter-app instead of flutter_app. Trigger and working directory repaired; remote workflow succeeds.
DEFECT-008 Load scripts use wrong credentials and weak auth assertion	Medium / High	Fixed and retested	Admin fixture password mismatched seeder; interview list accepted 401. Corrected fixture and 200-only assertion pass the live load run.
DEFECT-009 Uncommitted CV tool removal breaks AI imports	High / High	Isolated checkout restored and verified	User working copy removes get_application and get_application_resume, preventing collection. Verification uses the committed implementation; original open checkout remains untouched.

11 1 Reproduction and retesting of withdrawal

Preconditions: isolated PostgreSQL database, a published job, a registered Candidate with a profile and a Submitted application. Reproduce by POSTing to /api/Applications/{id}/withdraw using the owner JWT. Before repair, HTTP 500 was returned. API logs show EF sending UPDATE ApplicationHistory for a newly created row and PostgreSQL affecting zero rows. Repository graph traversal marked the client-generated history ID as an existing record.

ApplicationRepository.UpdateAsync now marks detached history entries Added and sets their ApplicationId before the repository updates the graph. The regression reloads a persisted application, adds withdrawal history, saves and verifies the state and one history row. The live retest independently confirms 204, a subsequent Withdrawn read and a history entry. Both backend regression and PostgreSQL checks pass.

Withdrawal defect before repair

Actual API test output | Expected HTTP 204 but received HTTP 500

```
{
  "id": "TC-LIVE-011",
  "name": "Candidate withdraws own application",
  "method": "POST",
  "path": "/Applications/61bb5a4e-28f7-4d1a-8104-98e355519e8d/withdraw",
  "input": null,
  "expected": "204",
  "actual": "HTTP 500",
  "status": "Fail",
  "duration_ms": 41.86,
  "detail": ""
}
```

Figure 16 Actual failing withdrawal check before the persistence repair

11 2 Prompt defect reproduction

For DEFECT-001, call `validate_prompt_safety` with a triple-backtick block containing New system instructions and an approval request. For DEFECT-002, use the supplied DAN and Bypass all restrictions prompt. The original baseline marks both assertions expected-fail. After the detector change, the xfail decorators are removed and both assertions pass normally. A zero-width-character variant of Ignore previous instructions is also required to block.

12 Test Execution Summary and Coverage

Suite	Executed	Passed	Failed	Evidence
Backend	56	56	0	backend.trx and cobertura XML
React	25	25	0	react.xml and verbose output
Flutter	16	16	0	flutter-results.jsonl and expanded output
AI	52	52	0	ai.xml and verbose output
Live API	22	22	0	live-api.json
PostgreSQL	5	5	0	database.log
Total	176	176	0	Unique automated cases and checks

Performance HTTP checks are intentionally excluded from the functional total. The baseline and final suite sizes differ because weak sample tests were replaced and production regressions were added. A higher count alone is not evidence of better coverage. The final pass rate for the selected 176 cases is 100 percent; it does not represent 100 percent feature, branch or line coverage.

12 1 Measured backend coverage

Instrumented assembly	Line coverage	Branch coverage
TalentFlow.Application	36.86 percent	50.00 percent
TalentFlow.Domain	41.04 percent	100.00 percent
TalentFlow.Infrastructure	12.05 percent	14.64 percent
Total instrumented code	1,585 / 11,568 lines = 13.70 percent	185 / 1249 branches = 14.81 percent

Coverage is generated by coverlet from the unit-test run. It includes Application, Domain and Infrastructure assemblies, and does not include the API assembly or UI code. The measured gap supports adding controller and authorization integration tests, completed offer and onboarding paths, and more persistence and failure scenarios. No React, Flutter or AI coverage percentage is claimed because those coverage exports were not collected.

12 2 Residual issues and priorities

Priority next steps are broader tenant isolation checks, an automated hire-to-onboarding path with controlled AI and calendar dependencies, actual mobile file upload and onboarding integration tests, and a security scan of a disposable deployment. Review Flutter informational analysis findings, the backend nullable-reference warning and the web bundle-size warning. Treat the prompt filter as a limited layer supported by server-side authorization and human approval.

13 Individual Responsibilities and Viva Demonstrations

The following ownership follows the supplied team assignments and component branches. It identifies who should review and demonstrate each contribution. Git author configuration records assigned responsibility; AI-assisted work must still be understood and explained personally. This report does not infer historic authorship from an uncommitted file or turn assigned responsibility into proof of a personal viva demonstration.

S H V A Rosa IT24100197

Assigned component: Company and Job Management.

Test sources and related changes: CompanyServiceTests.cs; existing JobServiceTests.cs; performance/k6/load_test_api.js and stress_test_auth.js.

Viva demonstration: Run company service tests; explain Moq repository behavior and missing-record assertions. Run a short k6 demonstration with controlled test credentials, then explain p95, request count and error thresholds.

Prescribed branch and exact commit are recorded in Section 14. All members should reproduce their assigned tests and explain the assertions, fixture boundaries, observed result and any defect before submission.

Hewapathirana S A IT24100097

Assigned component: Candidate and Application Management.

Test sources and related changes: ApplicationServiceTests.cs; ApplicationHistoryPersistenceTests.cs; ApplicationRepository.cs; React LoginPage and authStore tests; prompt-injection test cases.

Viva demonstration: Demonstrate the withdrawal regression and persisted history. Explain why a mocked repository missed a relational failure. Run React tests and the live API application scenario, including owner denial and duplicate rejection.

Prescribed branch and exact commit are recorded in Section 14. All members should reproduce their assigned tests and explain the assertions, fixture boundaries, observed result and any defect before submission.

Madhushan W M G IT24100715

Assigned component: Interview and Hiring Management.

Test sources and related changes: InterviewServiceTests.cs; OfferServiceTests.cs; Flutter production login, job list and router tests.

Viva demonstration: Demonstrate an interview duration boundary or invalid state. Run the real LoginScreen failure and navigation tests; explain fake AuthProvider and MockClient boundaries. Explain why an offer prerequisite test is not a completed offer workflow.

Prescribed branch and exact commit are recorded in Section 14. All members should reproduce their assigned tests and explain the assertions, fixture boundaries, observed result and any defect before submission.

Dewmini G D C P IT24102755

Assigned component: Employee and Onboarding Management.

Test sources and related changes: OnboardingServiceTests.cs; existing employee tests; production AI scoring tests; tool_permissions.py prompt repair.

Viva demonstration: Demonstrate forbidden onboarding assignment or missing task. Run scoring boundaries and prompt bypass regressions; explain allowlists, normalization and the need for human approval.

Prescribed branch and exact commit are recorded in Section 14. All members should reproduce their assigned tests and explain the assertions, fixture boundaries, observed result and any defect before submission.

14 GitHub Actions and Contribution Evidence

Repository: https://github.com/sishanhewa/SE3090_SE012. Four GitHub Actions workflows exist: Backend CI, React CI, Flutter CI and AI Service CI. The initial history showed backend successes, repeated React failures due to absent committed tests, and AI failures at Ruff. Flutter did not trigger for its actual directory because its path filter used a different folder spelling.

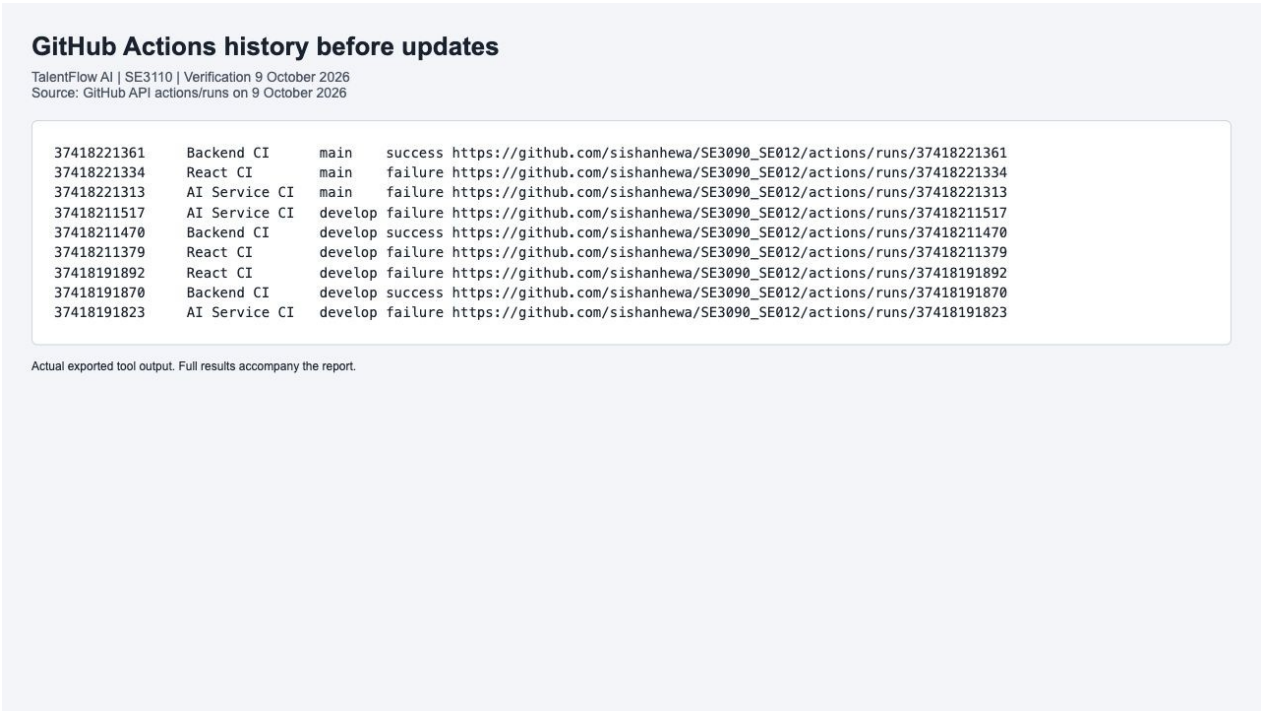


Figure 17 Actual GitHub Actions history retrieved before the testing updates

Shared CI repairs use develop and the supplied team lead identity sishanhewa with sishanhewa4@gmail.com. They correct Flutter paths, add workflow-change triggers and develop pull-request targets, repair AI lint blockers, upload web and AI XML results and use the actual JwtSettings configuration prefix. The shared commit 3983a74 was pushed successfully.

Workflow	Observed result	Run ID	Run link
React CI	success	37897709413	https://github.com/sishanhewa/SE3090_SE012/actions/runs/37897709413
AI Service CI	failure	37897302609	https://github.com/sishanhewa/SE3090_SE012/actions/runs/37897302609
Backend CI	success	37897302567	https://github.com/sishanhewa/SE3090_SE012/actions/runs/37897302567
Flutter CI	success	37896382097	https://github.com/sishanhewa/SE3090_SE012/actions/runs/37896382097

The shared CI repair made Backend, Flutter and AI workflows succeed. The lead application and React tests were subsequently merged through pull request 2 at commit 37367e4. Strict TypeScript checking then exposed two unused test imports, which were corrected and retested. AI safety tests require the prepared safety-owner implementation to accompany them. The table records the actual remote state at retrieval; it does not substitute for the complete local test run.

Prescribed branch	Configured author	Testing commit
develop	sishanhewa	3983a74d30f8

Prescribed branch	Configured author	Testing commit
feature/s1-company-jobs	vihara-rosa	6cfc9bb32605
feature/s2-candidates-apps	sishanhewa	b5caa824f59e
feature/s3-interviews-hiring	gaveeshamadhushan	755a7e8c569a
feature/s4-employees-onboarding	chenu222	3c3710eecb9f

Testing commit authors and component branches

Actual local Git history | Shared CI commit pushed; component pushes subject to account approval

```

develop
3983a74 sishanhewa <sishanhewa4@gmail.com>
fix(ci): repair test workflows and AI lint blockers

feature/s1-company-jobs
6cfc9bb vihara-rosa <rosavihara@gmail.com>
test(companies): add service cases and reproducible k6 load tests

feature/s2-candidates-apps
b5caa82 sishanhewa <sishanhewa4@gmail.com>
fix(applications): persist withdrawal history and add regression evidence

feature/s3-interviews-hiring
36e08c9 gaveeshamadhushan <gaveeshamadhushan15@gmail.com>
test(hiring): add interview offer and production mobile login checks

feature/s4-employees-onboarding
3c3710e chenu222 <dewminichethani222@gmail.com>
fix(ai-safety): block prompt bypasses and test production scoring rules

```

Figure 18 Actual component commit metadata and configured author identities

Only the lead GitHub account sishanhewa is authenticated in this environment. Lead pull requests https://github.com/sishanhewa/SE3090_SE012/pull/2 and https://github.com/sishanhewa/SE3090_SE012/pull/3 are merged. The second removes unused React test imports that blocked the strict TypeScript build. Member-specific commits are prepared using their exact configured names and emails on the prescribed feature branches after merging develop. Pushes through another account require the workflow exception requested from the project owner. The shared and lead-account work can be delivered with the existing authentication. The supporting delivery status file records the final remote state and pull-request links.

15 AI Assistance Declaration

AI assistance was used to audit the supplied draft, compare it with the assignment and reference report, inspect source and CI failures, develop and revise tests, diagnose and repair defects, generate the case register, collect evidence and edit this report. Test execution and result exports were checked against the actual TalentFlow code. No screenshots or numerical test results were generated as imagined outcomes.

AI assistance record	Declaration
Purpose and tool	OpenAI Codex supported the SE3110 evaluation of the existing SE3090 TalentFlow system, using the assignment brief and Group 020 report as a structural reference.
Assistance scope	Assistance supported test design, tooling, debugging, interpretation and report preparation.
Verification evidence	Actual TRX, XML, JSON, SQL, k6 exports and cursor-free browser captures support the recorded outcomes.
Student responsibility	Each member remains responsible for reviewing the assigned changes, understanding the tests and personally demonstrating them in the viva.
Reflection	Weak constant-only and generic-widget tests were replaced; low measured coverage and untested external integrations are explicitly identified.

This declaration records assistance, verification, responsibility and reflection. Each student remains responsible for following the module CLEAR requirements and acknowledging any additional AI tools used in their own work.

16 Conclusion

The final local verification supplies executable evidence for backend, database, web, mobile, integrated workflow, targeted security, AI and performance testing. All 176 selected automated cases passed after retesting, and both configured k6 scenarios met their thresholds. The most significant observed application defect was the withdrawal history persistence failure; its repair is verified by a regression and a live PostgreSQL workflow.

The results support the correctness of the tested behavior and provide a reproducible demonstration basis. They also expose clear remaining work: wider API and business-workflow coverage, live mobile integration, external-service evaluation and broader security assessment. Individual members must review and demonstrate their assigned work, and remote Git delivery must be read together with the account status and source commit of each CI run.

Appendix A Complete Test Case Register

The register below records every case executed in the final functional run. Preconditions are stated for each suite. Each record identifies the named automated scenario, execution/input or key expected assertion, actual outcome and status. The accompanying CSV retains the complete assertion text, source path and steps without shortening. Case IDs identify this verified run; they replace the draft register where its IDs or counts were inconsistent.

Backend cases

Preconditions: Fresh mocks or isolated EF InMemory context; build .NET 8 unit test project.

Execution: run the suite command in Section 4. Each row maps to the exact named test in the accompanying source. Inputs and fixtures are constructed in that test; the listed assertion is checked by the runner.

Case and scenario	Input or expected assertions	Actual result
TC-BE-001 Update Async Inserts New History And Persists Withdrawal	Expected assertions: Assert.Equal(ApplicationStatus.Withdrawn, (await repository.GetByIdAsync(application.Id)).Status); ; Assert.Equal(1, await context.ApplicationHistory.CountAsync(h => h.ApplicationId == application.Id));	Pass All assertions satisfied
TC-BE-002 Create Application Async Fail When Deadline Has Passed	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("deadline");	Pass All assertions satisfied
TC-BE-003 Create Application Async Fail When Duplicate Application	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("CONFLICT");	Pass All assertions satisfied
TC-BE-004 Create Application Async Fail When Job Is Closed	Expected assertions: result.IsSuccess.Should().BeFalse();	Pass All assertions satisfied
TC-BE-005 Create Application Async Return Bad Request When Job Is Not Published	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("applications");	Pass All assertions satisfied
TC-BE-006 Create Application Async Return Not Found When Job Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-007 Get Application By Id Async Return Not Found When Not Exists	Expected assertions: result.IsSuccess.Should().BeFalse();	Pass All assertions satisfied
TC-BE-008 Withdraw Application Async Return Not Found When Application Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-009 Calculate Score Non Existent Application Returns Not Found	Expected assertions: Assert.False(result.IsSuccess);	Pass All assertions satisfied
TC-BE-010 Score Breakdown Maximum Possible Is100	Expected assertions: Assert.Equal(100, mandatoryMax + preferredMax + experienceMax + educationMax + certMax);	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-BE-011 Create Company Async Handle Minimal Request	Expected assertions: result.IsSuccess.Should().BeTrue();	Pass All assertions satisfied
TC-BE-012 Create Company Async Return Success When Valid Request	Expected assertions: result.IsSuccess.Should().BeTrue(); ; result.Data.Should().NotNull(); ; result.Data!.Name.Should().Be("Tech Corp"); ; result.Data.IsActive.Should().BeTrue();	Pass All assertions satisfied
TC-BE-013 Create Company Async Set Is Active True	Expected assertions: capturedCompany.IsActive.Should().BeTrue();	Pass All assertions satisfied
TC-BE-014 Get Companies Async Return Paged Result	Expected assertions: result.IsSuccess.Should().BeTrue(); ; result.Data!.Items.Should().HaveCount(2);	Pass All assertions satisfied
TC-BE-015 Get Company By Id Async Return Company When Exists	Expected assertions: result.IsSuccess.Should().BeTrue(); ; result.Data!.Id.Should().Be(companyId); ; result.Data.Name.Should().Be("Existing Corp");	Pass All assertions satisfied
TC-BE-016 Get Company By Id Async Return Not Found When Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-017 Update Company Async Return Not Found When Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-018 Update Company Async Return Success When Company Exists	Expected assertions: result.IsSuccess.Should().BeTrue(); ; result.Data!.Name.Should().Be("New Name"); ; _companyRepositoryMock.Verify(repo => repo.UpdateAsync(It.IsAny<Company>(), It.IsAny<CancellationTokens>()), Times.Once);	Pass All assertions satisfied
TC-BE-019 Get Employee By Id Async Return Not Found When Employee Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-020 Configured Calendar Signs Token And Sends Candidate With Correct End Time	Expected assertions: Assert.True(result.IsSuccess); ; Assert.Contains("sendUpdates=all", handler.EventUri!.Query); ; Assert.Equal("tf1234567890abcdef1234567890abcdef", ; Assert.Equal(start.AddHours(1).ToString("o"), ; Assert.Equal("candidate@example.com",	Pass All assertions satisfied
TC-BE-021 Free Busy Slots Are Converted To Utc Before Conflict Checking	Expected assertions: Assert.True(result.IsSuccess); ; Assert.Equal(new DateTime(2026, 10, 8, 9, 30, 0, DateTimeKind.Utc), result.Data![0].Start);	Pass All assertions satisfied
TC-BE-022 Missing Credentials Never Report Invitation Or Availability As Successful	Expected assertions: Assert.False(invitation.IsSuccess); ; Assert.False(availability.IsSuccess);	Pass All assertions satisfied
TC-BE-023 Organizer OAuth Updates Calendar Only When Google Confirms Success	Expected assertions: Assert.True(confirmed.IsSuccess); ; Assert.Contains("grant_type=refresh_token", handler.TokenBody); ; Assert.Equal(start.AddHours(1).ToString("o"), ; Assert.False(failed.IsSuccess);	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-BE-024 Schedule Interview Async Fail When Active Interview Already Exists	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("CONFLICT");	Pass All assertions satisfied
TC-BE-025 Schedule Interview Async Fail When Application Not Shortlisted	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("Shortlisted");	Pass All assertions satisfied
TC-BE-026 Schedule Interview Async Fail When Duration Too Long	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("480");	Pass All assertions satisfied
TC-BE-027 Schedule Interview Async Fail When Duration Too Short	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("15");	Pass All assertions satisfied
TC-BE-028 Schedule Interview Async Fail When Scheduled In Past	Expected assertions: result.IsSuccess.Should().BeFalse();	Pass All assertions satisfied
TC-BE-029 Schedule Interview Async Reject Duration Below 15 Boundary Test	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("15");	Pass All assertions satisfied
TC-BE-030 Schedule Interview Async Return Not Found When Application Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-031 Create Job Async Return Success When Valid Request	Expected assertions: result.IsSuccess.Should().BeTrue(); ; result.Data.Should().NotNull(); ; result.Data!.Title.Should().Be(request.Title); ; result.Data.Status.Should().Be(JobStatus.Draft.ToString()); ; _jobRepositoryMock.Verify(repo => repo.AddAsync(It.Is<Job>(j => j.Title == request.Title), It.IsAny<CancellationToken>()), Times.Once);	Pass All assertions satisfied
TC-BE-032 Get Job By Id Async Return Not Found When Job Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-033 Create Offer Async Fail When Application Not In Interview Status	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("interview");	Pass All assertions satisfied
TC-BE-034 Create Offer Async Fail When Salary Is Negative	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.Error.Should().Contain("interview");	Pass All assertions satisfied
TC-BE-035 Create Offer Async Fail When Salary Is Zero	Expected assertions: result.IsSuccess.Should().BeFalse();	Pass All assertions satisfied
TC-BE-036 Create Offer Async Return Not Found When Application Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied
TC-BE-037 Get Offer By Id Async Return Not Found When Offer Does Not Exist	Expected assertions: result.IsSuccess.Should().BeFalse(); ; result.ErrorCode.Should().Be("NOT_FOUND");	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-BE-038 Get Offer By Id Async Return Offer When Exists	Expected assertions: result.IsSuccess.Should().BeTrue(); ; result.Data!.Position.Should().Be("Backend Developer");	Pass All assertions satisfied
TC-BE-039 Assign Onboarding Async Return Conflict When Already Assigned	Expected assertions: Assert.False(result.IsSuccess); ; Assert.Equal("CONFLICT", result.ErrorCode);	Pass All assertions satisfied
TC-BE-040 Assign Onboarding Async Return Forbidden When Company Mismatch	Expected assertions: Assert.False(result.IsSuccess); ; Assert.Equal("FORBIDDEN", result.ErrorCode);	Pass All assertions satisfied
TC-BE-041 Assign Onboarding Async Return Not Found When Employee Does Not Exist	Expected assertions: Assert.False(result.IsSuccess);	Pass All assertions satisfied
TC-BE-042 Assign Onboarding Async Return Not Found When Template Does Not Exist	Expected assertions: Assert.False(result.IsSuccess); ; Assert.Equal("NOT_FOUND", result.ErrorCode);	Pass All assertions satisfied
TC-BE-043 Complete Onboarding Task Async Mark Task As Completed	Expected assertions: Assert.True(result.IsSuccess); ; Assert.True(result.Data!.IsCompleted); ; Assert.NotNull(result.Data.CompletedAt); ; Assert.Equal("Completed reading", result.Data.Notes);	Pass All assertions satisfied
TC-BE-044 Complete Onboarding Task Async Return Not Found When Task Does Not Exist	Expected assertions: Assert.False(result.IsSuccess); ; Assert.Equal("NOT_FOUND", result.ErrorCode);	Pass All assertions satisfied
TC-BE-045 Create Template Async Persist Company Id	Expected assertions: Assert.True(result.IsSuccess); ; Assert.Equal(companyId, result.Data!.CompanyId);	Pass All assertions satisfied
TC-BE-046 Create Template Async Return Success With Valid Data	Expected assertions: Assert.True(result.IsSuccess); ; Assert.Equal("Standard Onboarding", result.Data!.Name); ; Assert.Equal(3, result.Data.Tasks.Count);	Pass All assertions satisfied
TC-BE-047 Get Templates By Company Async Return Empty When No Templates	Expected assertions: Assert.True(result.IsSuccess); ; Assert.Empty(result.Data!);	Pass All assertions satisfied
TC-BE-048 Get Available Slots Empty Schedule Returns Slots	Expected assertions: Assert.True(result.IsSuccess); ; Assert.NotEmpty(result.Data!);	Pass All assertions satisfied
TC-BE-049 Get Available Slots Uses Configured Business Time Zone Without Panel	Expected assertions: Assert.True(result.IsSuccess); ; Assert.NotEmpty(result.Data!); ; Assert.All(result.Data!, slot => ; ; Assert.InRange(local.Hour, 9, 16); ; Assert.NotEqual(DayOfWeek.Saturday, local.DayOfWeek); ; Assert.NotEqual(DayOfWeek.Sunday, local.DayOfWeek);	Pass All assertions satisfied
TC-BE-050 Has Candidate Conflict No Existing Interviews Returns False	Expected assertions: Assert.True(result.IsSuccess); ; Assert.False(result.Data);	Pass All assertions satisfied
TC-BE-051 Has Interviewer Conflict No Existing Interviews Returns False	Expected assertions: Assert.True(result.IsSuccess); ; Assert.False(result.Data);	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-BE-052 Approve Workflow Wrong Status Returns Fail	Expected assertions: <code>Assert.False(result.IsSuccess);</code>	Pass All assertions satisfied
TC-BE-053 Get Workflow Non Existent Returns Not Found	Expected assertions: <code>Assert.False(result.IsSuccess);</code>	Pass All assertions satisfied
TC-BE-054 Reject Workflow Records Human Decision And Rejects Application	Expected assertions: <code>Assert.True(result.IsSuccess); ; Assert.Equal(ApplicationStatus.Rejected, application.Status); ; Assert.Contains(context.ApplicationHistory, h => h.ApplicationId == application.Id && ; Assert.Contains(context.WorkflowApprovals, a => a.WorkflowExecutionId == workflow.Id &&</code>	Pass All assertions satisfied
TC-BE-055 Reject Workflow Requires Reason	Expected assertions: <code>Assert.False(result.IsSuccess); ; Assert.Equal(WorkflowStatus.AwaitingApproval, workflow.Status);</code>	Pass All assertions satisfied
TC-BE-056 Reject Workflow Wrong Status Returns Fail	Expected assertions: <code>Assert.False(result.IsSuccess);</code>	Pass All assertions satisfied

Source files: `backend/tests/TalentFlow.UnitTests/Services/ApplicationHistoryPersistenceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/ApplicationServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/CandidateScoringServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/CompanyServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/EmployeeServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/GoogleCalendarServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/InterviewServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/JobServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/OfferServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/OnboardingServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/SchedulingServiceTests.cs;`
`backend/tests/TalentFlow.UnitTests/Services/WorkflowServiceTests.cs`

React cases

Preconditions: JSDOM; reset auth mocks; MemoryRouter for login component tests.

Execution: run the suite command in Section 4. Each row maps to the exact named test in the accompanying source. Inputs and fixtures are constructed in that test; the listed assertion is checked by the runner.

Case and scenario	Input or expected assertions	Actual result
TC-WEB-001 should render the login form with email and password fields	Expected assertions: <code>expect(screen.getByLabelText(/email address/i)).toBeInTheDocument(); ; expect(screen.getByLabelText(/password/i)).toBeInTheDocument(); ; expect(screen.getByRole('button', { name: /sign in/i })).toBeInTheDocument();</code>	Pass All assertions satisfied
TC-WEB-002 should render the Talent Flow AI branding	Expected assertions: <code>expect(screen.getByText('TalentFlow')).toBeInTheDocument(); ; expect(screen.getByText('AI')).toBeInTheDocument(); ; expect(screen.getByText(/intelligent recruitment platform/i)).toBeInTheDocument();</code>	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-WEB-003 should display "Welcome back" heading	Expected assertions: expect(screen.getByText(/welcome back/i)).toBeInTheDocument();	Pass All assertions satisfied
TC-WEB-004 should have required attribute on email input	Expected assertions: expect(emailInput).toBeRequired();	Pass All assertions satisfied
TC-WEB-005 should have required attribute on password input	Expected assertions: expect(passwordInput).toBeRequired();	Pass All assertions satisfied
TC-WEB-006 should have email type on email input	Expected assertions: expect(emailInput).toHaveAttribute('type', 'email');	Pass All assertions satisfied
TC-WEB-007 should have password type on password input	Expected assertions: expect(passwordInput).toHaveAttribute('type', 'password');	Pass All assertions satisfied
TC-WEB-008 should update email field when user types	Expected assertions: expect(emailInput).toHaveValue('admin@talentflow.com');	Pass All assertions satisfied
TC-WEB-009 should update password field when user types	Expected assertions: expect(passwordInput).toHaveValue('securePass123');	Pass All assertions satisfied
TC-WEB-010 should call login function on form submit	Expected assertions: expect(mockLogin).toHaveBeenCalledWith('admin@talentflow.com', 'password123');	Pass All assertions satisfied
TC-WEB-011 should clear error before login attempt	Expected assertions: expect(mockClearError).toHaveBeenCalled();	Pass All assertions satisfied
TC-WEB-012 should display error message when error exists	Expected assertions: expect(screen.getByText('Invalid email or password')).toBeInTheDocument();	Pass All assertions satisfied
TC-WEB-013 should not display error message when no error	Expected assertions: expect(screen.queryByText(/invalid/i)).not.toBeInTheDocument();	Pass All assertions satisfied
TC-WEB-014 should show loading indicator when is Loading is true	Expected assertions: expect(screen.getByText(/signing in/i)).toBeInTheDocument();	Pass All assertions satisfied
TC-WEB-015 should disable submit button when loading	Expected assertions: expect(submitButton).toBeDisabled();	Pass All assertions satisfied
TC-WEB-016 should navigate to home on successful login	Expected assertions: expect(mockNavigate).toHaveBeenCalledWith('/');	Pass All assertions satisfied
TC-WEB-017 should not navigate when login fails	Expected assertions: expect(mockNavigate).not.toHaveBeenCalled();	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-WEB-018 should have correct initial state	Expected assertions: expect(state.accessToken).toBeNull(); ; expect(state.refreshToken).toBeNull(); ; expect(state.user).toBeNull(); ; expect(state.isAuthenticated).toBe(false); ; expect(state.isLoading).toBe(false); ; expect(state.error).toBeNull();	Pass All assertions satisfied
TC-WEB-019 should set loading state during login	Expected assertions: expect(useAuthStore.getState().isLoading).toBe(true); ; expect(useAuthStore.getState().error).toBeNull();	Pass All assertions satisfied
TC-WEB-020 should set auth data on successful login	Expected assertions: expect(state.accessToken).toBe('mock-access-token'); ; expect(state.refreshToken).toBe('mock-refresh-token'); ; expect(state.user).toEqual(mockUser); ; expect(state.isAuthenticated).toBe(true); ; expect(state.isLoading).toBe(false); ; expect(state.error).toBeNull();	Pass All assertions satisfied
TC-WEB-021 should set error on failed login	Expected assertions: expect(state.error).toBe('Invalid email or password'); ; expect(state.isAuthenticated).toBe(false); ; expect(state.isLoading).toBe(false); ; expect(state.accessToken).toBeNull();	Pass All assertions satisfied
TC-WEB-022 should set default error message when no server message	Expected assertions: expect(state.error).toBe('Login failed. Please check your credentials. '); ; expect(state.isLoading).toBe(false);	Pass All assertions satisfied
TC-WEB-023 should clear all auth data on logout	Expected assertions: expect(state.accessToken).toBeNull(); ; expect(state.refreshToken).toBeNull(); ; expect(state.user).toBeNull(); ; expect(state.isAuthenticated).toBe(false); ; expect(state.error).toBeNull();	Pass All assertions satisfied
TC-WEB-024 should clear error when clear Error is called	Expected assertions: expect(useAuthStore.getState().error).toBeNull();	Pass All assertions satisfied
TC-WEB-025 should call the correct API endpoint on login	Expected assertions: expect(apiClient.post).toHaveBeenCalled('Auth/login', {	Pass All assertions satisfied

Source files: frontend/react-app/src/__tests__/pages/LoginPage.test.tsx;
frontend/react-app/src/__tests__/store/authStore.test.ts

Flutter cases

Preconditions: Flutter widget test binding; real production screens; fake AuthProvider or MockClient; fresh preference store.

Execution: run the suite command in Section 4. Each row maps to the exact named test in the accompanying source. Inputs and fixtures are constructed in that test; the listed assertion is checked by the runner.

Case and scenario	Input or expected assertions	Actual result
TC-MOB-001 Unauthenticated users see the login screen	Expected assertions: expect(find.byType(LoginScreen), findsOneWidget);	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-MOB-002 Production jobs screen renders API job title and department	Expected assertions: expect(find.text('Available Jobs'), findsOneWidget); ; expect(find.text('Backend Engineer'), findsOneWidget); ; expect(find.text('Engineering'), findsNWidgets(2));	Pass All assertions satisfied
TC-MOB-003 Production login displays branding and input fields	Expected assertions: expect(find.text('TalentFlow'), findsOneWidget); ; expect(find.byType(TextField), findsNWidgets(2)); ; expect(find.text('Login'), findsOneWidget);	Pass All assertions satisfied
TC-MOB-004 Production router protects home when unauthenticated	Expected assertions: expect(find.byType(LoginScreen), findsOneWidget); ; expect(router.routeInformationProvider.value.uri.path, '/login');	Pass All assertions satisfied
TC-MOB-005 Production login masks the password	Expected assertions: expect(fields[0].keyboardType, TextInputType.emailAddress); ; expect(fields[1].obscureText, isTrue);	Pass All assertions satisfied
TC-MOB-006 Production login submits entered credentials	Expected assertions: expect(auth.email, 'candidate@example.test'); ; expect(auth.password, 'Evidence@123');	Pass All assertions satisfied
TC-MOB-007 Production router allows the public login route	Expected assertions: expect(find.byType(LoginScreen), findsOneWidget); ; expect(router.routeInformationProvider.value.uri.path, '/login');	Pass All assertions satisfied
TC-MOB-008 Production jobs screen displays empty API result	Expected assertions: expect(find.text('No jobs found'), findsOneWidget); ; expect(find.byType(CircularProgressIndicator), findsNothing);	Pass All assertions satisfied
TC-MOB-009 Production login displays authentication failure	Expected assertions: expect(find.text('Invalid email or password'), findsOneWidget);	Pass All assertions satisfied
TC-MOB-010 Production jobs screen safely completes failed API request	Expected assertions: expect(find.text('No jobs found'), findsOneWidget); ; expect(find.byType(CircularProgressIndicator), findsNothing); ; expect(tester.takeException(), isNull);	Pass All assertions satisfied
TC-MOB-011 Production login blocks repeated submission while pending	Expected assertions: expect(tester.widget<ElevatedButton>(find.byType(ElevatedButton)).onPressed, isNull); ; expect(find.byType(CircularProgressIndicator), findsOneWidget);	Pass All assertions satisfied
TC-MOB-012 Production jobs screen requests published jobs	Expected assertions: expect(requested!.path, '/api/jobs'); ; expect(requested!.queryParameters['Status'], '1');	Pass All assertions satisfied
TC-MOB-013 Production login re-enables submission after failure	Expected assertions: expect(tester.widget<ElevatedButton>(find.byType(ElevatedButton)).onPressed, isNotNull);	Pass All assertions satisfied
TC-MOB-014 Production login clears previous error during retry	Expected assertions: expect(find.text('Invalid email or password'), findsNothing);	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-MOB-015 Production jobs screen sorts newest jobs first	Expected assertions: expect(tester.getTopLeft(find.text('Newer role')).dy, ; lessThan(tester.getTopLeft(find.text('Older role')).dy));	Pass All assertions satisfied
TC-MOB-016 Production login navigates home after success	Expected assertions: expect(find.text('Verified home'), findOneWidget);	Pass All assertions satisfied

Source files: frontend/flutter_app/test/jobs_list_test.dart; frontend/flutter_app/test/login_screen_test.dart; frontend/flutter_app/test/navigation_test.dart; frontend/flutter_app/test/widget_test.dart

AI cases

Preconditions: Python dependencies installed; deterministic input; external HTTP/Gemini replaced where applicable.

Execution: run the suite command in Section 4. Each row maps to the exact named test in the accompanying source. Inputs and fixtures are constructed in that test; the listed assertion is checked by the runner.

Case and scenario	Input or expected assertions	Actual result
TC-AI-001 test validate scoring valid	Expected assertions: assert result["is_valid"] is True ; assert len(result["errors"]) == 0	Pass All assertions satisfied
TC-AI-002 test validate scoring wrong total	Expected assertions: assert result["is_valid"] is False ; assert any("does not equal" in e for e in result["errors"])	Pass All assertions satisfied
TC-AI-003 test validate scoring wrong recommendation	Expected assertions: assert result["is_valid"] is False ; assert any("NotRecommended" in e for e in result["errors"])	Pass All assertions satisfied
TC-AI-004 test validate scoring out of range	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in e for e in result["errors"])	Pass All assertions satisfied
TC-AI-005 test validate application state valid	Expected assertions: assert result["is_valid"] is True	Pass All assertions satisfied
TC-AI-006 test validate application state wrong status	Expected assertions: assert result["is_valid"] is False ; assert any("Screening" in e for e in result["errors"])	Pass All assertions satisfied
TC-AI-007 test validate schema missing fields	Expected assertions: assert result["is_valid"] is False ; assert any("job_id" in str(e) for e in result["errors"])	Pass All assertions satisfied
TC-AI-008 test validate authorization correct role	Expected assertions: assert result["is_valid"] is True	Pass All assertions satisfied
TC-AI-009 test validate authorization wrong company	Expected assertions: assert result["is_valid"] is False ; assert any("company" in e.lower() for e in result["errors"])	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-AI-010 test allowed tool	Expected assertions: assert check_tool_permission(	Pass All assertions satisfied
TC-AI-011 test blocked tool	Expected assertions: with pytest.raises(ToolPermissionError) as exc_info: ; assert "blocked" in str(exc_info.value).lower()	Pass All assertions satisfied
TC-AI-012 test unauthorized tool	Expected assertions: with pytest.raises(ToolPermissionError):	Pass All assertions satisfied
TC-AI-013 test prompt safety clean	Expected assertions: assert result["safe"] is True	Pass All assertions satisfied
TC-AI-014 test prompt safety injection	Expected assertions: assert result["safe"] is False ; assert len(result["detected_patterns"]) > 0	Pass All assertions satisfied
TC-AI-015 test candidate analysis output schema	Expected assertions: assert output.application_id == "app-1" ; assert len(output.mandatory_skill_matches) == 1 ; assert output.meets_experience_requirement is True	Pass All assertions satisfied
TC-AI-016 test validation output schema	Expected assertions: assert output.is_valid is True ; assert output.risk_level == "Low" ; assert len(output.warnings) == 1	Pass All assertions satisfied
TC-AI-017 test workflow plan schema	Expected assertions: assert len(plan.steps) == 2 ; assert plan.steps[1].depends_on == [1]	Pass All assertions satisfied
TC-AI-018 test production recommendation boundaries[100-Eligible For Interview-parts0]	Expected assertions: assert result["is_valid"] is True ; assert invalid["is_valid"] is False	Pass All assertions satisfied
TC-AI-019 test production recommendation boundaries[75-Eligible For Interview-parts1]	Expected assertions: assert result["is_valid"] is True ; assert invalid["is_valid"] is False	Pass All assertions satisfied
TC-AI-020 test production recommendation boundaries[74-Manual Review-parts2]	Expected assertions: assert result["is_valid"] is True ; assert invalid["is_valid"] is False	Pass All assertions satisfied
TC-AI-021 test production recommendation boundaries[60-Manual Review-parts3]	Expected assertions: assert result["is_valid"] is True ; assert invalid["is_valid"] is False	Pass All assertions satisfied
TC-AI-022 test production recommendation boundaries[59-Not Recommended-parts4]	Expected assertions: assert result["is_valid"] is True ; assert invalid["is_valid"] is False	Pass All assertions satisfied
TC-AI-023 test production recommendation boundaries[0-Not Recommended-parts5]	Expected assertions: assert result["is_valid"] is True ; assert invalid["is_valid"] is False	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-AI-024 test production component range guards[mandatory Skills Score-41]	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-025 test production component range guards[preferred Skills Score-21]	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-026 test production component range guards[experience Score-26]	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-027 test production component range guards[education Score-11]	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-028 test production component range guards[certification Score-6]	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-029 test production component range guards[mandatory Skills Score--1]	Expected assertions: assert result["is_valid"] is False ; assert any("out of range" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-030 test production total must equal component sum	Expected assertions: assert result["is_valid"] is False ; assert any("component sum" in error for error in result["errors"])	Pass All assertions satisfied
TC-AI-031 test workflow result defaults to human approval	Expected assertions: assert result.requires_approval is True	Pass All assertions satisfied
TC-AI-032 test candidate analysis requires application id	Expected assertions: with pytest.raises(ValidationError):	Pass All assertions satisfied
TC-AI-033 test validation output requires validity decision	Expected assertions: with pytest.raises(ValidationError):	Pass All assertions satisfied
TC-AI-034 test workflow plan requires steps	Expected assertions: with pytest.raises(ValidationError):	Pass All assertions satisfied
TC-AI-035 test attached docx is read and used for skill matching	Expected assertions: assert url.endswith("/applications/app-1/resume") ; assert headers["Authorization"] == "Bearer recruiter-token" ; assert [match.is_matched for match in matches] == [True, False]	Pass All assertions satisfied
TC-AI-036 test unreadable docx fails instead of screening empty evidence	Expected assertions: with pytest.raises(ValueError, match="No readable text"):	Pass All assertions satisfied
TC-AI-037 test screening recommendation comes from cv checks not calendar slots[True-Shortlist]	Expected assertions: assert result.screening_recommendation == expected ; assert result.requires_approval is True	Pass All assertions satisfied

Case and scenario	Input or expected assertions	Actual result
TC-AI-038 test screening recommendation comes from cv checks not calendar slots[False-Do Not Shortlist]	Expected assertions: assert result.screening_recommendation == expected ; assert result.requires_approval is True	Pass All assertions satisfied
TC-AI-039 test ignore previous instructions	Expected assertions: assert result["safe"] is False ; assert len(result["detected_patterns"]) > 0	Pass All assertions satisfied
TC-AI-040 test system prompt override	Expected assertions: assert result["safe"] is False	Pass All assertions satisfied
TC-AI-041 test role override attempt	Expected assertions: assert result["safe"] is False	Pass All assertions satisfied
TC-AI-042 test data exfiltration attempt	Expected assertions: assert "safe" in result	Pass All assertions satisfied
TC-AI-043 test instruction injection with delimiters	Expected assertions: assert result["safe"] is False	Pass All assertions satisfied
TC-AI-044 test jailbreak do anything now	Expected assertions: assert result["safe"] is False	Pass All assertions satisfied
TC-AI-045 test sql injection in candidate query	Expected assertions: assert "safe" in result	Pass All assertions satisfied
TC-AI-046 test safe prompt analysis request	Expected assertions: assert result["safe"] is True	Pass All assertions satisfied
TC-AI-047 test safe prompt scoring request	Expected assertions: assert result["safe"] is True	Pass All assertions satisfied
TC-AI-048 test safe prompt interview request	Expected assertions: assert result["safe"] is True	Pass All assertions satisfied
TC-AI-049 test safe prompt with technical terms	Expected assertions: assert result["safe"] is True	Pass All assertions satisfied
TC-AI-050 test empty prompt	Expected assertions: assert result["safe"] is True	Pass All assertions satisfied
TC-AI-051 test very long prompt	Expected assertions: assert "safe" in result	Pass All assertions satisfied
TC-AI-052 test unicode injection	Expected assertions: assert result["safe"] is False ; assert result["action"] == "blocked"	Pass All assertions satisfied

Source files: agentic-ai/tests/test_agents.py; agentic-ai/tests/test_business_rules.py;
agentic-ai/tests/test_cv_screening.py; agentic-ai/tests/test_prompt_injection.py

Live API cases

Preconditions: Isolated seeded PostgreSQL database and running API; create two synthetic candidate accounts; retain returned IDs and JWTs in memory.

Case and scenario	Input or expected assertions	Actual result
TC-LIVE-001 Seeded admin authenticates	Steps: POST /Auth/login ; input: credentials redacted Expected: 200 and body assertions	Pass HTTP 200
TC-SEC-001 Protected companies reject anonymous access	Steps: GET /Companies ; input: None Expected: 401	Pass HTTP 401
TC-SEC-002 Malformed JWT is rejected	Steps: GET /Auth/me ; input: credentials redacted Expected: 401	Pass HTTP 401
TC-SEC-003 Wrong password is rejected	Steps: POST /Auth/login ; input: credentials redacted Expected: 401	Pass HTTP 401
TC-SEC-004 Public staff-role escalation is rejected	Steps: POST /Auth/register ; input: credentials redacted Expected: 400	Pass HTTP 400
TC-SEC-005 Malformed login input is rejected	Steps: POST /Auth/login ; input: credentials redacted Expected: 400	Pass HTTP 400
TC-LIVE-002 Register isolated candidate 1	Steps: POST /Auth/register ; input: credentials redacted Expected: 201 and body assertions	Pass HTTP 201
TC-LIVE-003 Create candidate profile 1	Steps: POST /CandidateProfiles ; input: {'summary': 'SE3110 isolated integration verification'} Expected: 201 and body assertions	Pass HTTP 201
TC-LIVE-004 Register isolated candidate 2	Steps: POST /Auth/register ; input: credentials redacted Expected: 201 and body assertions	Pass HTTP 201
TC-LIVE-005 Create candidate profile 2	Steps: POST /CandidateProfiles ; input: {'summary': 'SE3110 isolated integration verification'} Expected: 201 and body assertions	Pass HTTP 201
TC-SEC-006 Candidate cannot create company	Steps: POST /Companies ; input: {'name': 'Unauthorized evidence company'} Expected: 403	Pass HTTP 403
TC-LIVE-006 Public jobs can be browsed	Steps: GET /jobs ; input: None Expected: 200 and body assertions	Pass HTTP 200
TC-LIVE-007 Candidate submits to published job	Steps: POST /Applications/jobs/9fd61350-85e1-4aaa-b4a9-64714b2a0e32 ; input: {'coverLetter': 'SE3110 live workflow evidence'} Expected: 201 and body assertions	Pass HTTP 201
TC-LIVE-008 Candidate sees persisted application	Steps: GET /Applications/5e4be457-f637-4252-930d-3b858db248f2 ; input: None Expected: 200 and body assertions	Pass HTTP 200
TC-SEC-007 Other candidate cannot read application	Steps: GET /Applications/5e4be457-f637-4252-930d-3b858db248f2 ; input: None Expected: 403	Pass HTTP 403

Case and scenario	Input or expected assertions	Actual result
TC-SEC-008 Candidate cannot list staff applications	Steps: GET /Applications ; input: None Expected: 403	Pass HTTP 403
TC-LIVE-009 Duplicate application is rejected	Steps: POST /Applications/jobs/9fd61350-85e1-4aaa-b4a9-64714b2a0e32 ; input: {'coverLetter': 'Duplicate'} Expected: 409	Pass HTTP 409
TC-LIVE-010 Staff sees submitted application	Steps: GET /Applications/5e4be457-f637-4252-930d-3b858db248f2 ; input: None Expected: 200 and body assertions	Pass HTTP 200
TC-LIVE-011 Candidate withdraws own application	Steps: POST /Applications/5e4be457-f637-4252-930d-3b858db248f2/withdraw ; input: None Expected: 204	Pass HTTP 204
TC-LIVE-012 Staff sees persisted withdrawal	Steps: GET /Applications/5e4be457-f637-4252-930d-3b858db248f2 ; input: None Expected: 200 and body assertions	Pass HTTP 200
TC-LIVE-013 Withdrawal is in application history	Steps: GET /Applications/5e4be457-f637-4252-930d-3b858db248f2/history ; input: None Expected: 200 and body assertions	Pass HTTP 200
TC-SEC-009 Other candidate cannot withdraw application	Steps: POST /Applications/5e4be457-f637-4252-930d-3b858db248f2/withdraw ; input: None Expected: 403	Pass HTTP 403

Source files: docs/testing/scripts/live_api_tests.py

PostgreSQL cases

Preconditions: Run live API workflow first; psql connection to isolated verification database.

Case and scenario	Input or expected assertions	Actual result
TC-DB-001 Duplicate job candidate pair	Steps: Execute database_checks.sql with ON_ERROR_STOP; assertions raise an exception on unexpected behavior; rollback all test mutations. Expected: PostgreSQL unique_violation prevents duplicate application	Pass PASS notice emitted; psql exit code 0
TC-DB-002 Orphan application history	Steps: Execute database_checks.sql with ON_ERROR_STOP; assertions raise an exception on unexpected behavior; rollback all test mutations. Expected: PostgreSQL foreign_key_violation rejects unknown ApplicationId	Pass PASS notice emitted; psql exit code 0
TC-DB-003 Null application status	Steps: Execute database_checks.sql with ON_ERROR_STOP; assertions raise an exception on unexpected behavior; rollback all test mutations. Expected: PostgreSQL not_null_violation rejects NULL status	Pass PASS notice emitted; psql exit code 0
TC-DB-004 Persisted withdrawal and history	Steps: Execute database_checks.sql with ON_ERROR_STOP; assertions raise an exception on unexpected behavior; rollback all test mutations. Expected: Join finds Withdrawn application with Withdrawn history	Pass PASS notice emitted; psql exit code 0

Case and scenario	Input or expected assertions	Actual result
TC-DB-005 Rollback restores previous values	Steps: Execute database_checks.sql with ON_ERROR_STOP; assertions raise an exception on unexpected behavior; rollback all test mutations. Expected: CoverLetter sentinel does not survive rollback to savepoint	Pass PASS notice emitted; psql exit code 0

Source files: docs/testing/scripts/database_checks.sql

Appendix B Evidence Inventory and References

Evidence inventory

Artifact	Purpose
SE3110_SE012_Test_Cases.csv	All 176 cases with complete preconditions, steps, expected outcome, actual outcome, status and source path
backend.trx and coverage.cobertura.xml	56 final backend results and measured line/branch coverage
react.xml and react.log	25 web test records and verbose output
flutter-results.jsonl and flutter-final.log	16 production Flutter test records and output
ai.xml and ai-final.log	52 final AI test records; baseline prompt failure log retained separately
live-api.json and live-api-before-fix.json	22 final API checks and the pre-repair withdrawal failure
database.log and database_checks.sql	PostgreSQL constraint and transaction assertions and reproducible SQL
load-summary.json and stress-summary.json	Actual k6 metrics and threshold results
screenshots	Cursor-free captures of the actual TalentFlow UI and saved tool outputs
automated test source and workflow files	Source needed to rerun the documented tests
delivery-status.md	Final remote branch and pull-request state; local and remote source scope

List of figures

- 1 TalentFlow admin dashboard after signing in to the isolated verification system
- 2 Actual test source locations and React runner configuration
- 3 Actual backend test output with 56 passes and generated coverage attachment
- 4 Actual PostgreSQL assertion notices, rollback output, version and applied migrations
- 5 Real TalentFlow login page before entering credentials
- 6 Actual Vitest output for LoginPage and authStore with 25 passes
- 7 Real TalentFlow job list loaded from the verification API
- 8 Actual Flutter output from the revised production widget and router suite
- 9 Actual live API execution output with 22 passing workflow and security checks
- 10 Withdrawn candidate application displayed in the real TalentFlow administration interface
- 11 Supplied prompt-safety baseline showing the two documented expected failures
- 12 Final AI tool output after prompt repairs and meaningful scoring tests
- 13 Actual k6 load-test metric values and threshold results
- 14 Actual k6 authentication stress-test metric values and threshold results
- 15 Measured concurrency and response-time trends from the k6 time-series exports
- 16 Actual failing withdrawal check before the persistence repair

17 Actual GitHub Actions history retrieved before the testing updates

18 Actual component commit metadata and configured author identities

References

SE3110 Assignment PDF, Software Testing and Quality Evaluation of the SE3090 Integrated System, four pages, supplied course brief.

TalentFlow AI SE3090_SE012 repository and the supplied working-tree test sources, inspected and executed on 9 October 2026.

SE3110 Assignment Group020 SoftwareTestingReport PDF, 104 pages. Used to compare report structure, individual demonstration content and evidence organization; no results or screenshots were reused.

SE3110 Assignment SE012 SoftwareTestingReport DOCX, supplied AI-generated draft. Revised claims were checked against actual source and execution evidence.

priority_instructions.md and .agents/AGENTS.md in the project, used for exact configured authors and prescribed branches.

GitHub Actions run links and exact source commits listed in Section 14.